WARN/ERROR Query Anchor

이전 로그의 evidence 를 더 높은 순위에 두는가

평가 설계 1-5 / 다중 evidence ranking / 핵심 식

비교 설계 / 지표 / 통계 / 판정 / 거리 분석

1. 같은 후보에서 Anchor 와 baseline 을 비교한다

평가 단위

WARN/ERROR 한 줄을 query 로 정하고
그보다 앞선 line 만 후보로 둔다 .
실제 evidence line 이 정답이다 .

Recency score(k) = −(Anchor line − k)

All-Query score(k) = 평균 Attention(q → k), k < q ≤ Anchor

Position-matched INFO · 선택 control

WARN/ERROR 의 추가 query 신호를 확인

Recency · 필수 control

정답이 가까운 line 이라 높은 순위인지 확인

All-Query · Main baseline

한 줄 집중 전략의 이점을 따로 확인

같은 후보 ID 와 정답 ID 를 사용한다 . INFO 가 볼 수 없는 후속 line 은 두 query 모두의 후보에서 제외한다 .

2. MAP 은 모든 evidence 의 순위를 반영한다

주 지표 : MAP

정답 evidence 가 여러 줄이면
각 정답이 발견되는 순위를 모두 평가한다 .

예시

정답 순위 : 1 등 , 4 등

AP = (P@1 + P@4) / 2 = (1.00 + 0.50) / 2 = 0.75

AP(i) = (P@r1 + P@r2 + … + P@rR) / R

MAP = 평균 [incident 별 평균 (anchor 별 AP)]

P@r = r 위까지의 정답 비율 ・ r1…rR = evidence 가 나온 순위 ・ R = evidence 개수

정답이 1 개면 AP = 1 / 정답 순위 = RR. 여러 anchor 는 먼저 incident 안에서 평균낸다 .

2. Top-K 보존에는 Recall 과 Precision 을 사용한다

Recall@5 / @10

전체 정답 중 상위 K 줄에 포함된 비율이다 .

Recall@K = Top-K 안에서 찾은 evidence 수 / 전체 evidence 수

Precision@5 / @10

상위 K 줄 가운데 정답이 차지하는 비율이다 .

Precision@K = Top-K 안에서 찾은 evidence 수 / K

예 : evidence 가 5 개이고 Top-10 에 4 개가 있으면 Recall@10 = 4/5 = 0.80, Precision@10 = 4/10 = 0.40.

정답 수가 적으면 Precision@K 의 최대값도 낮다 . K=5 와 10 은 실제 보존 예산에 맞게 정한 설정이다 .

2. MRR 과 AUC 는 순위의 다른 면을 본다

MRR

첫 정답의 순위만 반영한다 . 여러 정답의 나머지 순위는 포함하지 않는다 .

RR(i) = 1 / 첫 번째 evidence 의 순위

MRR = RR 의 평균 (anchor → incident 순으로 평균)

Evidence AUC / PRS

정답 하나가 비정답보다 높은 점수를 받는 비율이다 . 점수가 같으면 0.5 를 준다 .

Pairwise score(e) = (이긴 non-evidence 수 + 0.5× 동점 수) / 전체 non-evidence 수

Evidence AUC = 모든 evidence 의 Pairwise score 평균

Pairwise score(e) 는 evidence e 가 non-evidence 보다 얼마나 위에 있는지 0~1 로 나타낸다 . AUC=0.5 는 random 수준이다 .

정답이 1 개면 AP=RR. AUC 와 평균 pairwise score 는 같은 정보를 나타내므로 별도 성공 근거로 중복 계산하지 않는다 .

3. 차이와 불확실성을 incident 단위로 본다

효과 크기

d_c(B) = incident c 의 Anchor AP 평균 − baseline B 의 AP 평균

ΔMAP(B) = 모든 incident 의 d_c(B) 평균

95% 신뢰구간과 permutation

95% CI = bootstrap ΔMAP 의 2.5% 지점 ~ 97.5% 지점

p-value ≈ |ΔMAP_perm| ≥ |ΔMAP_obs| 인 permutation 의 비율

APbar_c 는 사건 c 에 속한 anchor 의 AP 평균이다 . 사건을 통째로 재표집하는 bootstrap 10,000 회와 양측 permutation 100,000 회를 사용한다 .

D 는 ΔMAP 이다 . All-Query 와 Recency 두 주요 비교의 p-value 는 Holm 보정한다 . 한 incident 의 여러 anchor 를 독립 관측치로 보지 않는다 .

4. GO 는 두 주요 비교를 모두 통과해야 한다

① ΔMAP(All-Query) > 0 AND ΔMAP(Recency) > 0

② Holm 보정 p(All-Query) < .05 AND p(Recency) < .05

GO

위 두 조건을 통과한다 .
Recall@5/10 과 거리별 방향도 확인 .

Conditional GO

한 비교에서만 우위이거나 ,
특정 dataset 에서만 효과 .

NO-GO / 보류

우위를 확인하지 못한 상태 .
큰 CI 는 효과 없음의 증명이 아니다 .

Position-matched INFO 보다도 좋다면 WARN/ERROR 라는 query 특성의 추가 이점을 보조적으로 확인할 수 있다 .

5. 먼 evidence 에서도 효과가 유지되는가

Distance(e) = Anchor line ID − Evidence line ID

연속 거리

정답별 pairwise score 가 거리가 멀어져도 유지되는지 본다 .

분석용 거리 = log(1 + Distance)

Near / Mid / Far

dev set 에서 정한 경계 두 개를 test 에 그대로 적용한다 .

ΔPairwise(d) = Anchor pairwise score(d) − Baseline pairwise score(d)

바로 앞의 evidence 에서만 좋다면 근접성 효과일 수 있다 . 먼 구간의 incident 수와 불확실성도 함께 보고한다 .

2 층 평가 (효율 / 비용) 구체화

11 주차 · part_D · " 같은 품질을 더 적은 자원으로 " 를 어떻게 공정하게 재나

2. 무엇을 정확히 재나

지표 · 측정 방법 · 단위

지표	측정 방법	단위
KV 메모리	이론 계산 + GPU 피크 실측	GB
TTFT	실측 (time.time)	ms
throughput	실측	tokens/sec
inference time	실측	s

KV 메모리 공식 2 × 레이어수 × KV 헤드수 × head_dim × 토큰수 × 바이트수

※ 시간 계열 (TTFT·throughput·inference) 은 실측만 가능 . KV 메모리만 공식 이론 계산도 → 둘 다 보고 . torch.cuda.synchronize() 로 GPU 작업 끝 시점을 잡아야 시간이 정확 . (담당 : 민규님)

3. 핵심 지표

원본 원인 (G) 단위당 절감

```
원인당 메모리 절감 = ( 무압축 KV 메모리 − 압축 KV 메모리 ) / |G|
원인당 지연 절감 = ( 무압축 지연 − 압축 지연 ) / |G|
```

- 분모 |G| = 원본의 전체 원인 라인 수 (그라운드 트루스). ' 압축 후 보존한 수 ' 가 아님 .
- 이유 : 분모가 모든 압축기에 동일한 고정값이어야 공정 . ' 각자 보존한 수 ' 로 하면 분모가 달라져 비교 불가 + 덜 보존한 쪽이 유리해 보이는 역설 발생 .
- **→ 명칭도 'retained evidence 단위당 ' 이 아니라 ' 원본 원인 (G) 단위당 ' 이 정확 .**

4. 어텐션 추출 비용 처리

forward 는 공통 비용 → 소거

- baseline(SnapKV·H2O) 도 우리도 어텐션을 위해 forward 를 1 회 수행 → forward 는 양쪽 공통 비용이라 비교에서 소거 .
- 비교 대상 = forward 위에 각자 추가로 드는 연산만 . 우리 추가분 = 로그신호 (레벨 · 희귀도 · 위치) 계산인데 파싱 · 카운트라 forward 대비 미미 .

```
순절감 = ( 압축으로 아낀 메모리 · 지연 ) − (forward 위에 추가로 쓴 비용 )
         └ forward 자체는 양쪽 공통이라 이 식에서 소거됨
```

⚠ **예외** output sensitivity 재려고 라인마다 재 -forward 하면 그건 공통 아님 → 그 부분만 별도 계산 .

5. 합격 기준

파레토 우위 — 안 지고 하나는 확실히 이기기

- 보존율과 비용을 같이 놓고 본다 . 둘 다 baseline 보다 나쁘지 않으면서 , 최소 하나에서 확실히 이기면 성공 .

이기는 3 가지 경우 (다 성공)

① 더 잘 보존 + 더 쌈 → 완승

② 보존율 같음 + 더 쌈 → 성공

강조용 : " 원인 보존율 X% 를 맞추는 데 우리는 압축률 Y% 까지 가능한데 baseline 은 Z% 밖에 못 간다 = 같은 품질을 더 싸게 ." / 통계 보조 : 부트스트랩 신뢰구간으로 흔들림 확인 .

③ 비용 같음 + 더 보존 → 성공